UNIT 14 80 min

Classes and Objects

Bundle data and behavior together: define classes, create objects, write methods, and use inheritance.

By the end of this unit you will be able to

- Define a class and create objects (instances) from it
- Write `__init__` to set up each object, and explain `self`
- Write methods that read and change an object's attributes
- Tell class attributes from instance attributes
- Create a subclass that reuses and extends a parent class with `super()`
- Use a dataclass for a simple record

Why classes

So far, data and functions have been separate: a dictionary holds a student's name and grades, and a function computes the average from it. A **class** combines the two. It defines a kind of object, with data (**attributes**) and behavior (**methods**) kept together. Every value you have used, such as a list or a string, is already an object of some class.

A first class

student.py

```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        self.grades.append(grade)

    def average(self):
        return sum(self.grades) / len(self.grades)

maya = Student("Maya")
maya.add_grade(91)
maya.add_grade(88)
print(maya.name, maya.average())
```

- `class Student:` defines the blueprint. By convention, class names are CapitalizedWords.
- `__init__` runs once when you create an object. It sets up the starting attributes.
- `self` is the object the method was called on. It is always the first parameter of a method, and Python fills it in for you.
- `Student("Maya")` creates an **instance**. Each instance has its own separate data.

Trace: two objects, separate data TRACE

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return self.name + " says woof"

a = Dog("Rex")
b = Dog("Milo")
print(a.bark())
print(b.name)
```

Step	Line run	Variables after the line	Output so far
1	5	`Dog = class`	`—`
2	8	`Dog = class, a = Dog(name="Rex")`	`—`
3	9	`Dog = class, a = Dog(name="Rex"), b = Dog(name="Milo")`	`—`
4	10	`Dog = class, a = Dog(name="Rex"), b = Dog(name="Milo")`	`Rex says woof`
5	11	`Dog = class, a = Dog(name="Rex"), b = Dog(name="Milo")`	`Rex says woof | Milo`

Class attributes and instance attributes

An attribute set on `self` belongs to one object. An attribute set directly in the class body belongs to the class, and every object shares it. Use class attributes for constants or counters that describe the whole group.

class_attr.py

```
class Course:
    school = "Denmark High School"   # shared by every Course

    def __init__(self, title):
        self.title = title           # one per object

python = Course("Python")
ai = Course("AI")
print(python.school, "|", ai.title)
```

Special methods: __str__

Methods named with double underscores on both sides are **special methods**. Python calls them automatically. `__str__` controls what `print()` shows for your object.

str_method.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)
print(str(p))
```

Inheritance

A **subclass** inherits everything from a parent class and can add or replace methods. Write `class Child(Parent):`. Call `super().__init__()` to run the parent's setup before adding your own.

inheritance.py

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

class Cat(Animal):
    def speak(self):
        return f"{self.name} says meow"

class Cow(Animal):
    def speak(self):
        return f"{self.name} says moo"

for pet in [Cat("Ivy"), Cow("Bess")]:
    print(pet.speak())
```

Both subclasses replace `speak`. This is **polymorphism**: the loop does not care which kind of animal it has, because each one knows how to speak.

Dataclasses: records without boilerplate

When a class is only a bundle of fields, the `dataclasses` module writes `__init__` and `__repr__` for you. `@dataclass` is a **decorator**: a line above the class that changes it.

dataclass.py

```
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    pages: int

b = Book("Dune", 412)
print(b)
print(b.pages > 300)
```

Tip

Use a plain class when behavior matters most. Use a dataclass when the object is mostly data, such as a row of a table.

Predict the output THINK FIRST

Trace what each object holds and what each method returns.

```
class C:
    def __init__(self, n):
        self.n = n

print(C(5).n)
```

Answer: `5`

```
class C:
    count = 0
    def __init__(self):
        C.count += 1

C()
C()
print(C.count)
```

Answer: `2`
The shared class attribute counts every object created.

```
class A:
    def hi(self):
        return "A"
class B(A):
    pass
print(B().hi())
```

Answer: `A`
B has no hi of its own, so it inherits A's.

```
class P:
    def __init__(self, x):
        self.x = x
    def __str__(self):
        return f"P{self.x}"
print(P(7))
```

Answer: `P7`

Practice

▼ **Exercise 1 — Bank account**

Write a class `Account` with an opening balance set in `__init__`, a `deposit(amount)` method, and a `withdraw(amount)` method that refuses (returns `False`) when the balance is too low. Deposit 100, withdraw 30, then try to withdraw 500 and print the final balance.

▼ **Hint**

Check `amount <= self.balance` before subtracting.

▼ **Exercise 2 — Shapes**

Create a class `Rectangle` with `width` and `height`, a method `area()`, and a subclass `Square` whose constructor takes one size. Print the area of a `Square(4)`.

Discussion

1. What is the difference between a class and an object? Use a real-world example that is not a computer.
2. When a method uses data from many different parts of a program, is it a sign that the data should belong to a class? Why or why not?
3. Inheritance can make code shorter or harder to follow. When would you avoid it?

Key terms

Class

A blueprint that defines the attributes and methods shared by a kind of object.

Object / instance

A specific value created from a class. Each instance has its own attributes.

Attribute

A variable that belongs to an object, read with a dot, such as `maya.name`.

Method

A function defined inside a class. It receives the object as `self`.

self

The conventional name of the first parameter of a method, referring to the object the method was called on.

__init__

The special method that sets up a new object when it is created.

__str__

The special method that returns the text shown by `print()` and `str()`.

Inheritance

A class that takes over the attributes and methods of a parent class, written `class Child(Parent):`.

super()

A way to call a method from the parent class, such as the parent's `__init__`.

Polymorphism

Using one interface on different kinds of objects, where each responds in its own way.

Dataclass

A class decorated with `@dataclass` that generates the constructor and printable form from its field annotations.

Decorator

A line starting with `@` placed above a function or class that wraps or modifies it.

See all terms in the glossary →